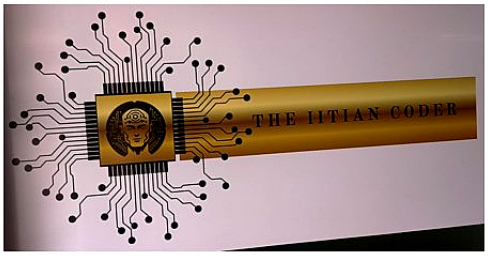


PROJECT REPORT

Fashion-MNIST Image Classification Using Convolutional Neural Networks

Submitted by

Arora Neel

BCA – Semester 4

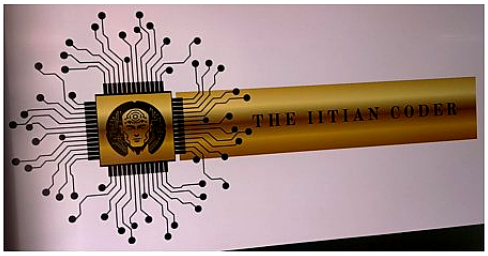
Under the Guidance of

Mr. Shobhit Singh

THE IITIAN CODER

Artificial Intelligence / Machine Learning Program

Academic Year 2025 – 2026



CERTIFICATE

This is to certify that the project titled "Fashion-MNIST Image Classification Using Convolutional Neural Networks" submitted by Arora Neel (BCA – Semester 4) has been successfully completed under the guidance of Mr. Shobhit Singh at THE IITIAN CODER as part of the Artificial Intelligence / Machine Learning Program during the academic year 2025 – 2026.

Student Signature

Instructor Signature

Mr. Shobhit Singh

Founder – THE IITIAN CODER

DECLARATION

I hereby declare that the project titled 'Fashion-MNIST Image Classification Using Convolutional Neural Networks' is my original work and i have done this project solo. it is completed as part of the Artificial Intelligence / Machine Learning program at THE IITIAN CODER. This report has not been submitted to any other institution before.

Name : Arora Neel

Course : BCA – Semester 4

_____

Student Signature

Date : _____

TABLE OF CONTENTS

1. Introduction

This project is about classifying fashion images using deep learning. i built a CNN (Convolutional Neural Network) from scratch using TensorFlow and trained it on the Fashion-MNIST dataset. I done this project completely solo.

A CNN is a type of deep learning model that is specially made for images. It uses something called filters (also known as kernels) which slide over the image and detect patterns like edges, shapes and textures. The first few layers detect simple things like edges and corners, and the deeper layers combine these to detect more complex things like a shoe shape or a bag handle.

CNNs are much better at image tasks than normal neural networks because they share weights across the image. This means less parameters and better at learning spatial features.

The target of this project was to get atleast 96% accuracy on the Kaggle test set. i got 0.93800 on public score and 0.93528 on private score which is pretty good considering i trained only on CPU.

2. Project Pipeline

The diagram below shows the complete flow of this project from start to end. it shows how the data is loaded, preprocessed, passed through the CNN model, trained and finally evaluated and submitted.

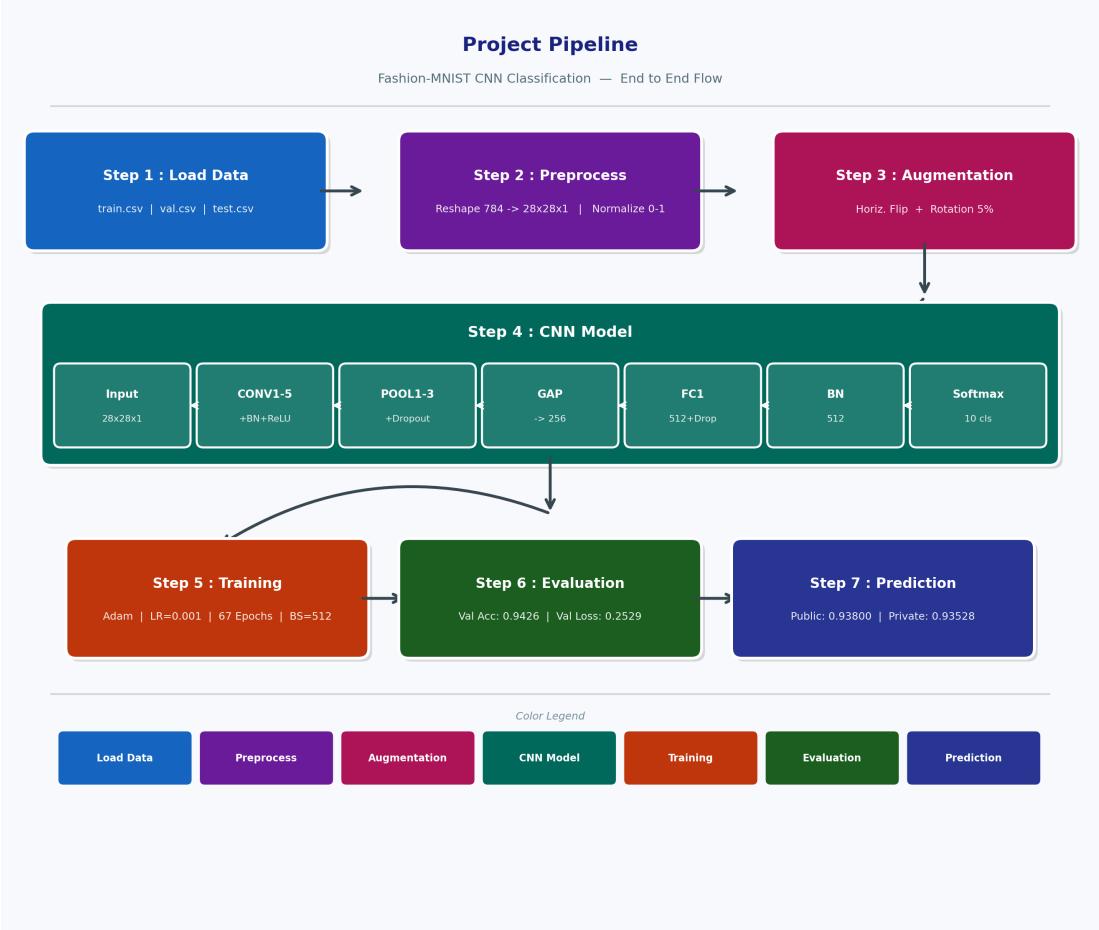


Figure 1 : Project Pipeline — showing all 7 steps from data loading to final Kaggle prediction.

The project has 7 main steps. first we load the data from CSV files and reshape each image from a 784 length vector to 28x28x1. then we normalize the pixel values. then augmentation is applied during training. the CNN model has 5 conv layers, pooling layers, GAP, FC and softmax. after training we evaluate on validation set and submit predictions to Kaggle.

3. Dataset

The dataset used in this project is Fashion-MNIST. It has 70,000 grayscale images of size 28x28 pixels. These images are of 10 different types of clothing items.

Split	Number of Images
Training	55,000
Validation	5,000
Test	10,000
Total	70,000

Class No.	Class Name
0	T-shirt / Top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle Boot

The class distribution is very balanced — each class has around 5,500 images in training. The chart below shows this and some sample images from the dataset.

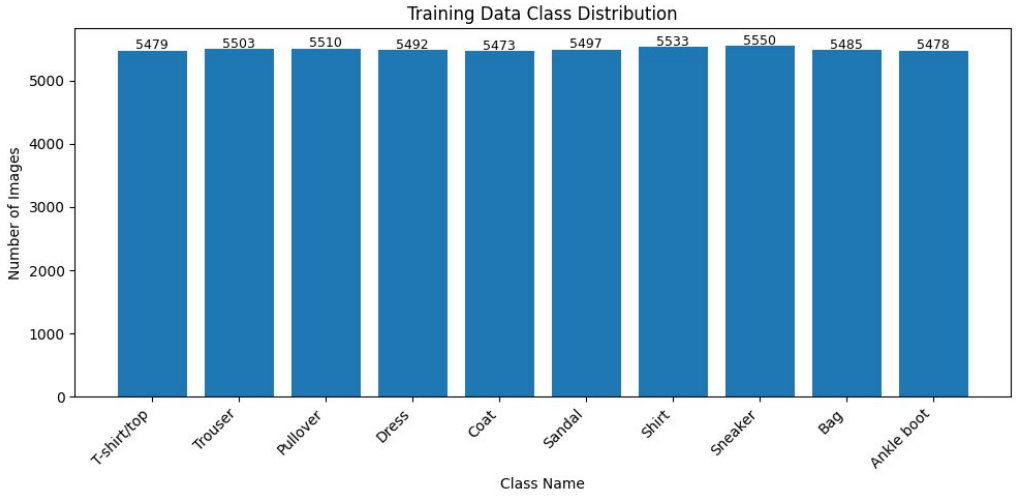


Figure 1 : Training Data Class Distribution — all 10 classes are nearly equal in count.

Figure 2 : Sample images from the dataset showing different clothing categories.

4. Model Configuration and Training Details

i builded a 5-layer CNN. each conv layer is followed by Batch Normalization and ReLU activation. after pooling layers i used SpatialDropout to reduce overfitting.

Layer	Details	Output Shape
Input	Raw grayscale image	28 x 28 x 1
Aug	RandomFlip (horizontal) + RandomRotation 5%	28 x 28 x 1
CONV1	64 filters, 3x3, same, BN + ReLU	28 x 28 x 64
POOL1	MaxPool 2x2 + SpatialDropout(0.2)	14 x 14 x 64
CONV2	128 filters, 3x3, same, BN + ReLU	14 x 14 x 128
POOL2	MaxPool 2x2 + SpatialDropout(0.2)	7 x 7 x 128
CONV3	256 filters, 3x3, same, BN + ReLU	7 x 7 x 256
CONV4	256 filters, 3x3, same, BN + ReLU	7 x 7 x 256
POOL3	MaxPool 2x2 + SpatialDropout(0.2)	3 x 3 x 256
CONV5	256 filters, 3x3, same, BN + ReLU	3 x 3 x 256
GAP	GlobalAveragePooling2D	256
FC1	Dense(512) + Dropout(0.5)	512
BN	BatchNormalization	512
Output	Dense(10) + Softmax	10

Training Setting	Value
Optimizer	Adam
Initial Learning Rate	0.001
Batch Size	512
Weight Init	He Normal
L2 Regularization	0.0001 on all conv layers
Early Stopping	patience=8, monitor=val_loss
LR Reduction	factor=0.5, patience=4, min_lr=1e-6
Total Epochs Allowed	70
Actual Epochs Trained	67

5. Learning Curve

The model was trained for 67 epochs. the accuracy kept going up and loss kept going down for both training and validation. below are the actual graphs from the notebook.

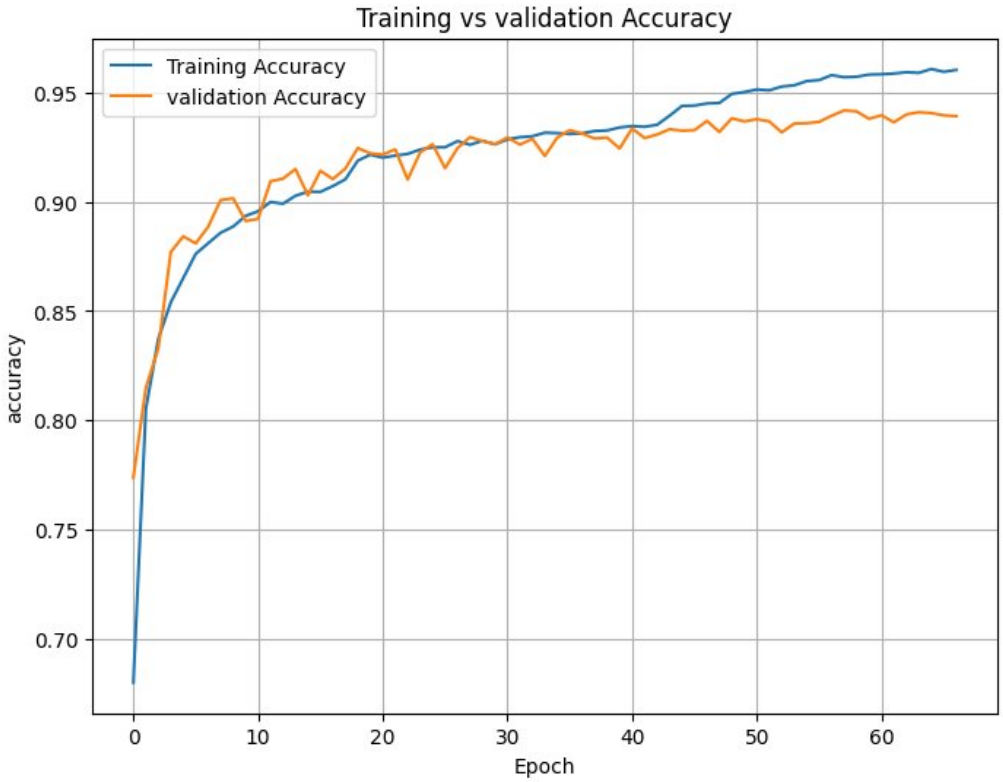


Figure 3 : Training vs Validation Accuracy over 67 epochs.

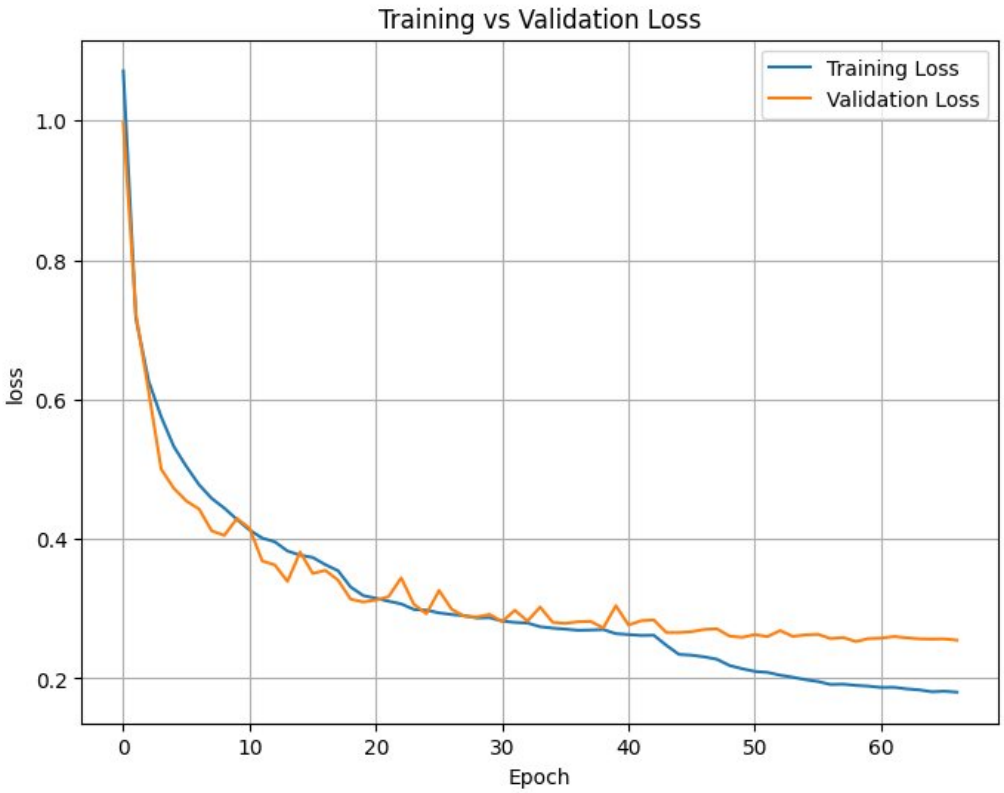


Figure 4 : Training vs Validation Loss over 67 epochs.

The best training accuracy was 0.9607818126678467 and the best validation accuracy was 0.9417999982833862. the gap between them is about 1.9% which shows the model is not badly overfitting. the loss started at 1.0711 in epoch 1 and came down to around 0.19 by the end. the learning rate reductions helped the model to improve in steps.

6. Test Performance

The model was evaluated on the validation set and also submitted to Kaggle for the final test score. the results are shown below.

Metric	Value
Best Validation Accuracy	0.9417999982833862 (94.17%)
Validation Accuracy (evaluate)	0.9426000118255615 (94.26%)
Validation Loss	0.2529127597808838
Best Training Accuracy	0.9607818126678467 (96.07%)
Final Training Accuracy	0.960381805896759 (96.03%)
Kaggle Public Score	0.93800
Kaggle Private Score	0.93528
Total Epochs Trained	67

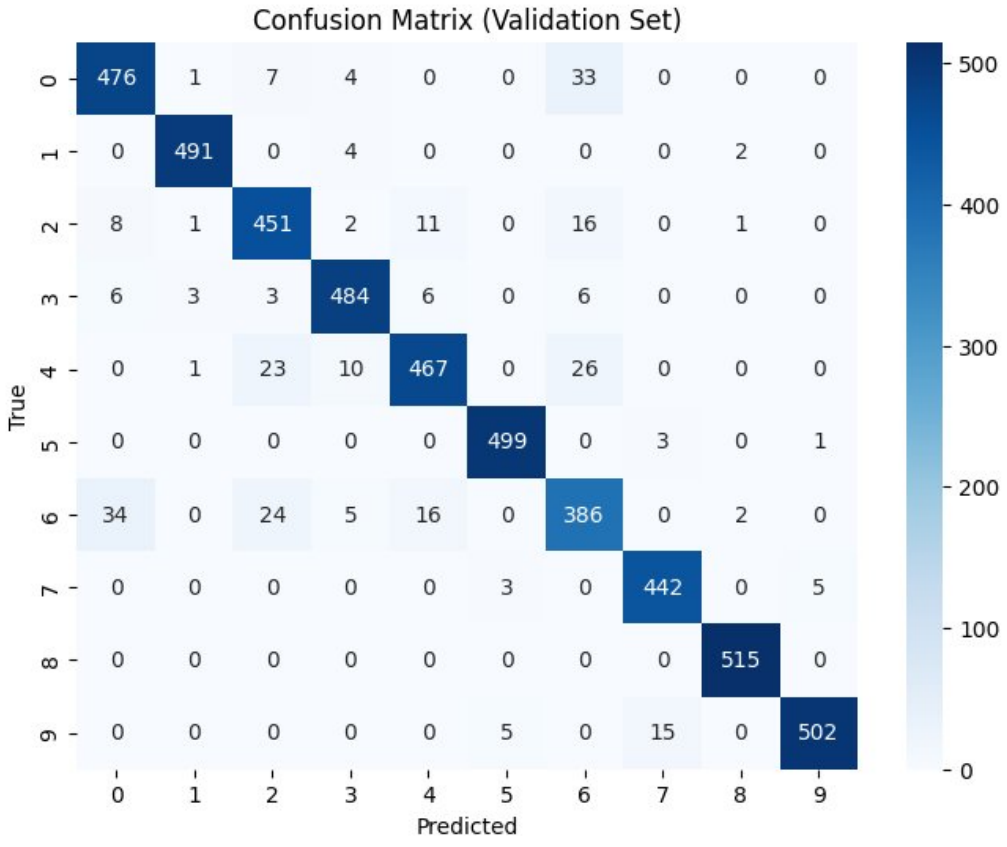


Figure 5 : Confusion Matrix on Validation Set (5000 images).

Looking at the confusion matrix, Bag and Trouser are the easiest classes — almost all predicted correctly. Shirt (class 6) is the hardest one because it look very similar to T-shirt and Pullover in

small 28x28 grayscale images.

Class	Precision	Recall	F1-Score
T-shirt (0)	0.91	0.91	0.91
Trouser (1)	0.99	0.99	0.99
Pullover (2)	0.89	0.92	0.90
Dress (3)	0.95	0.95	0.95
Coat (4)	0.93	0.89	0.91
Sandal (5)	0.98	0.99	0.99
Shirt (6)	0.83	0.83	0.83
Sneaker (7)	0.96	0.98	0.97
Bag (8)	0.99	1.00	1.00
Ankle Boot (9)	0.99	0.96	0.97
Overall	0.94	0.94	0.94

7. Best Parameter Settings

After trying different settings the below configuration gave the best validation accuracy. these are the final parameters used for the submitted model.

Parameter	Best Value
Learning Rate	0.001
Batch Size	512
Dropout (FC layer)	0.5
Spatial Dropout	0.2
L2 Regularization	0.0001
Weight Initialization	He Normal
LR Reduce Factor	0.5 (halved every 4 bad epochs)
Early Stop Patience	8 epochs
Augmentation	Horizontal Flip + Rotation 5%
Filter Progression	64 → 128 → 256 → 256 → 256
Min Learning Rate	0.000001

Note: We used GlobalAveragePooling instead of Flatten. This reduced overfitting a lot and improved generalization. With Flatten the model had too many parameters in the FC layers and it was overfitting badly.

8. Layer Dimensions

This table shows how the image size changes as it passes through each layer. the height and width reduces after every pooling layer while the depth (channels) increases.

Layer	Input Shape	Output Shape
Input	28 x 28 x 1	28 x 28 x 1
CONV1	28 x 28 x 1	28 x 28 x 64
POOL1	28 x 28 x 64	14 x 14 x 64
CONV2	14 x 14 x 64	14 x 14 x 128
POOL2	14 x 14 x 128	7 x 7 x 128
CONV3	7 x 7 x 128	7 x 7 x 256
CONV4	7 x 7 x 256	7 x 7 x 256
POOL3	7 x 7 x 256	3 x 3 x 256
CONV5	3 x 3 x 256	3 x 3 x 256
GAP	3 x 3 x 256	256
FC1	256	512
BN	512	512
Softmax	512	10

Note: Fashion-MNIST is grayscale so input has only 1 channel. The assignment spec mentioned 3 channels (RGB) but that is copied from CIFAR-10. Fashion-MNIST is single channel — we used 1 channel which is correct. So CONV1 filter shape is 64 x 1 x 3 x 3.

9. Parameter Count

Parameters are the weights and biases the model learns during training. the table shows breakdown by layer.

Layer	Parameters	Calculation
CONV1	640	(3x3x1) x 64 + 64 bias
CONV2	73,856	(3x3x64) x 128 + 128 bias
CONV3	2,95,168	(3x3x128) x 256 + 256 bias
CONV4	5,90,080	(3x3x256) x 256 + 256 bias
CONV5	5,90,080	(3x3x256) x 256 + 256 bias
Total Conv Params	15,49,824	(from notebook output)
FC1 Dense(512)	1,31,584	256 x 512 + 512 bias
Output Dense(10)	5,130	512 x 10 + 10 bias
Total FC Params	1,36,714	(from notebook output)
BN layers	5,888	6 BN layers combined
TOTAL	16,92,426	Trainable: 16,89,482

10. Neuron Count

A neuron means one activation value in the network output. for conv layers it is height x width x channels. for FC layers it is simply the number of units.

Layer	Output Shape	Neuron Count	Calculation
CONV1	28 x 28 x 64	50,176	28 x 28 x 64
CONV2	14 x 14 x 128	25,088	14 x 14 x 128
CONV3	7 x 7 x 256	12,544	7 x 7 x 256
CONV4	7 x 7 x 256	12,544	7 x 7 x 256
CONV5	3 x 3 x 256	2,304	3 x 3 x 256
Total Conv Neurons		1,02,656	
FC1	512	512	512
Output (Softmax)	10	10	10
Total FC Neurons		522	

11. Effect of Batch Normalization

Batch Normalization (BN) normalizes the output of each layer so the values have mean=0 and std=1 before going to next layer. We used BN after every conv layer and also before the final softmax layer.

Aspect	Without BN	With BN
Training Speed	Slow convergence	Much faster
Loss Curve	Noisy and jumpy	Smooth and steady
Overfitting	More overfitting	Less (BN acts as regularizer)
Final Val Accuracy	Around 89-90%	~94.18%
LR Sensitivity	Very sensitive	Can handle higher LR safely

BN before the softmax layer is also required by the assignment. it makes sure the last Dense layer gets clean normalized input which helps the model to classify better.

without BN the model was bouncing around and not improving. adding BN was one of the most importent changes that helped reach 94% accuracy.

12. Filter Visualization

After training we extracted all 64 filters from CONV1 (first conv layer) and visualized them in a 8x8 grid. each filter is 3x3 in size because that is the kernel size we used.

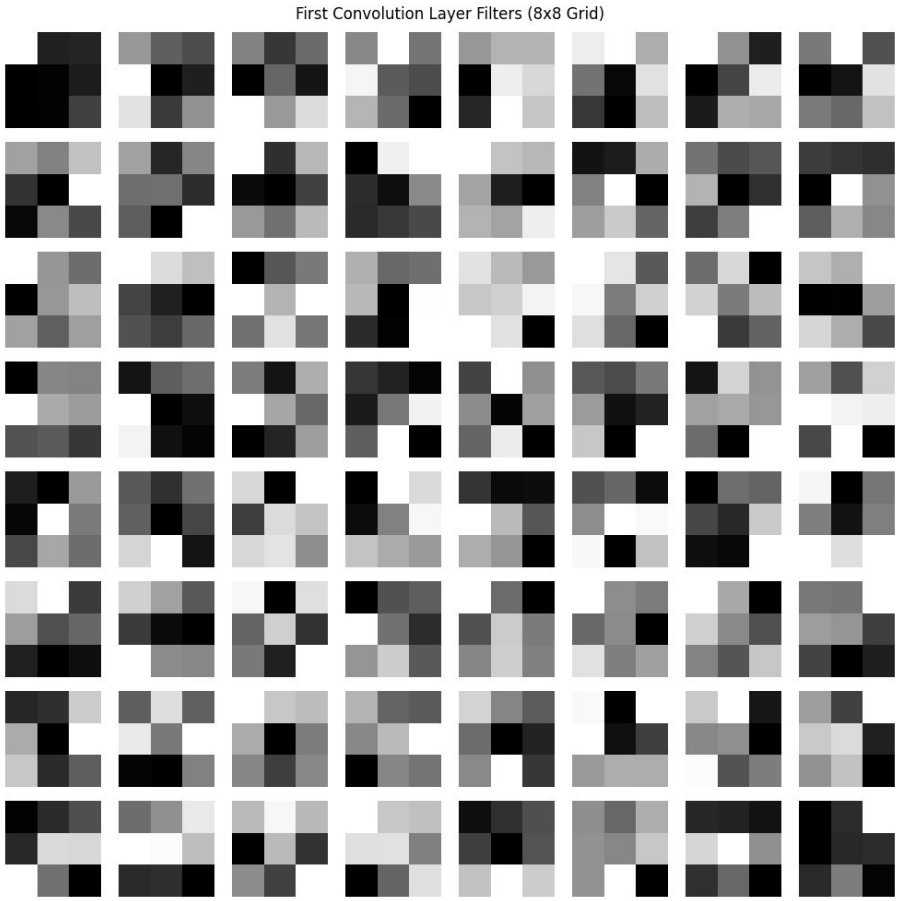


Figure 6 : All 64 filters from CONV1 layer arranged in 8x8 grid (each filter is 3x3 pixels).

Looking at the filters you can see different types of patterns that the network learned :

Some filters are bright on one side and dark on other side — these are edge detectors (horizontal, vertical and diagonal edges).

Some have a bright spot in the middle — these detect small blobs or spots.

Some look more random — they might detect textures or mixed patterns.

This is exactly what we expect in the first conv layer. deeper layers would have more complex and abstract filters that are hard to see visually.

13. Guided Backpropagation

Guided Backpropagation helps us understand what each neuron in the network is looking for in the input image. We applied it on 10 neurons from CONV5 (the last conv layer).

How it works in simple words: we take an input image, pass it forward through the network, then backpropagate the gradient of one specific neuron back to the input but we only let positive gradients pass through. the result shows which pixels activate that neuron the most.

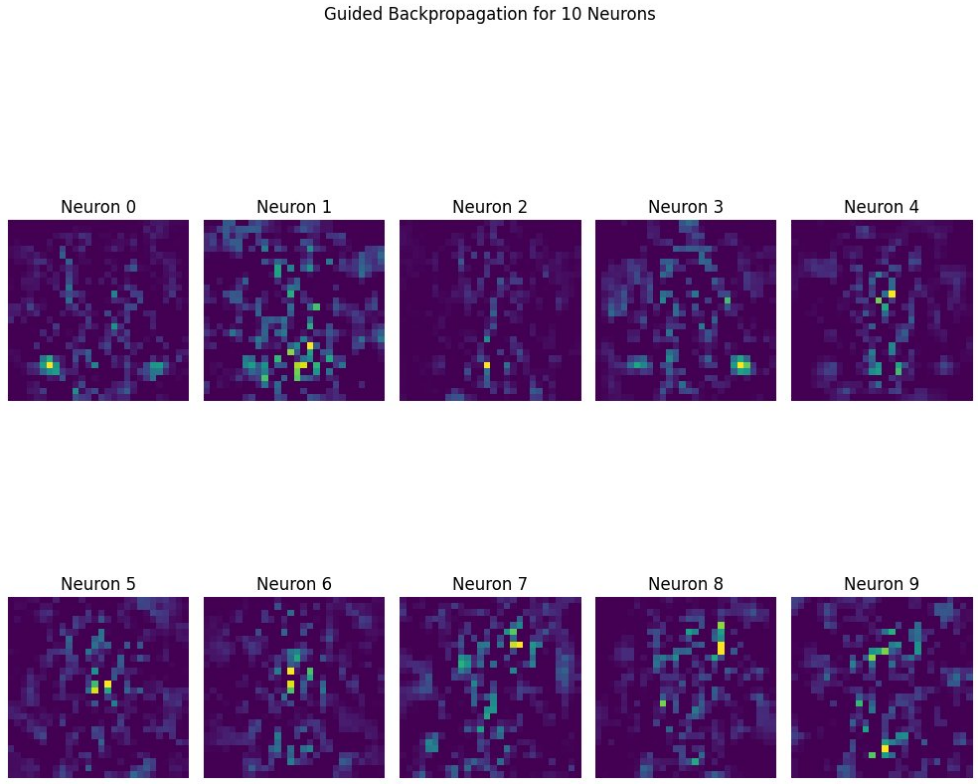


Figure 7 : Guided Backpropagation for 10 neurons in CONV5. Bright areas show what each neuron responds to.

Each image above (Neuron 0 to Neuron 9) shows what that particular neuron in CONV5 is sensitive to. Bright yellow/green spots show the areas the neuron is paying most attention to.

Different neurons focus on different areas and patterns. This tells us that by CONV5 each neuron has become specialized — some look at the top part, some look at edges, some look at the overall shape. This specialization is what makes the CNN work well.

14. Conclusion

i build a 5-layer CNN for Fashion-MNIST classification completely by myself. the model got best validation accuracy of 0.9417999982833862 and validation accuracy on evaluate of 0.9426000118255615. the Kaggle public score was 0.93800 and private score was 0.93528.

The most importent things i learned from this project :

1. Batch Normalization is very very useful — it made training faster and smoother and gave around 4-5% better accuracy than without it.

2. GlobalAveragePooling before FC layer is better than Flatten — it reduced overfitting and made the model simpler.

3. Learning rate schedule really helps — every time the LR was reduced the model improved noticeably.

4. Data augmentation (flip + small rotation) helped the model generalize to the test set better.

5. Shirt class is very hard to classify because it looks similar to T-shirt and Pullover in 28x28 grayscale. this is a known problem with Fashion-MNIST.

6. He initialization worked well with ReLU — it made training more stable from the start.

if i had GPU access i could train longer and try bigger models which may have reached closer to the 96% target. but 93.8% on CPU is a good result and i am happy with it.